



CSOC WG

AWS Cost Savings



Intro

Focus: Reducing Gen3 operating costs.

Goal: Run multiple secure, FedRAMP-compliant AWS environments for under \$1,000/month.

Status: Platform Engineering is actively iterating and making steady progress toward this target.



Agenda

Karpenter

Compute provisioning

Cilium

Overlay networking

Shrinking Portal

Build once - serve many!

AWS Cost Usage Reports

Quick tips on how to use them?

Karpenter + Spot Instances for Cost Savings

Goal: Reduce AWS compute costs (~70%) for Kubernetes workloads

Technologies:



Karpenter: Dynamically provisions right-sized nodes



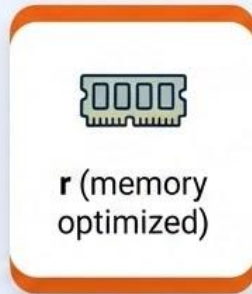
Spot Instances: Low-cost, interruptible compute

Strategy:

- Use Spot capacity
- Enable flexible instance categories (c / m / r)
- Apply consolidation policies
- Ensure uptime with Replicas & PDBs

Instance Categories (Flexibility = Savings)

- Enabled multiple instance families:



Result:

- ✓ Higher chance of getting Spot capacity
- ✓ Lower cost via flexible scheduling



Avoided locking into specific instance types

Enforcing Spot Usage (Karpenter)

Configuration



```
- key: karpenter.sh/capacity-type  
  operator: In  
  values:  
    - spot
```



Benefits:



Ensures workloads only run on Spot nodes



Prevents accidental use of on-demand capacity

Instance Sizing Constraints



```
- key: karpenter.k8s.aws/instance-memory
  operator: Gt
  values:
    - '7000'
- key: karpenter.k8s.aws/instance-memory
  operator: Lt
  values:
    - '32500'
```

- Restricts nodes to ~7GB – 32GB range
- Avoids:
 - Very small inefficient nodes
 - Very large expensive nodes

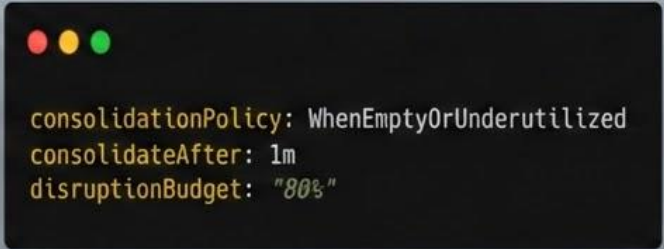


Better cost vs utilization balance



Consolidation (Major Cost Saver)

- **Removes:**
 - Empty nodes
 - Underutilized nodes
- **Replaces nodes with cheaper alternatives**



```
consolidationPolicy: WhenEmptyOrUnderutilized  
consolidateAfter: 1m  
disruptionBudget: "80%"
```

👉 **Aggressive optimization with controlled disruption**

Handling Spot Interruptions (Uptime)

- Spot instances **can be terminated anytime**.



What we could implement:



Replicas for all workloads



Pod Disruption Budgets (PDBs)



Workload separation:



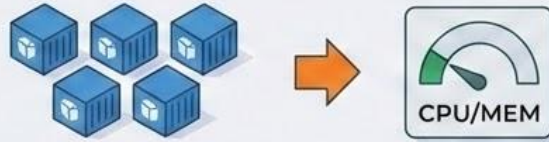
Critical → On-demand

Non-critical → Spot



Ensures *availability* despite interruptions

Cilium – The problem



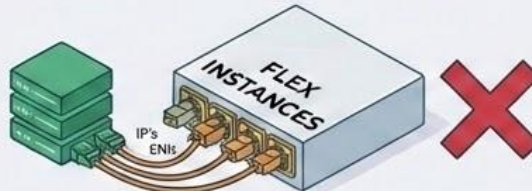
PODS OPTIMIZED: Reduced CPU/Memory requests.



POD DENSITY LIMIT REACHED



CAUSE: EC2 instance limits on ENI's and real IP's.



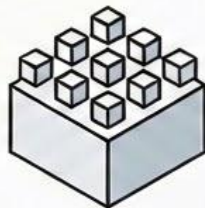
STILL LIMITED BY POD DENSITY: Flex instances not enough.



Cilium - The solution



Cilium uses Virtual IP addresses for pod networking, not host IPs.

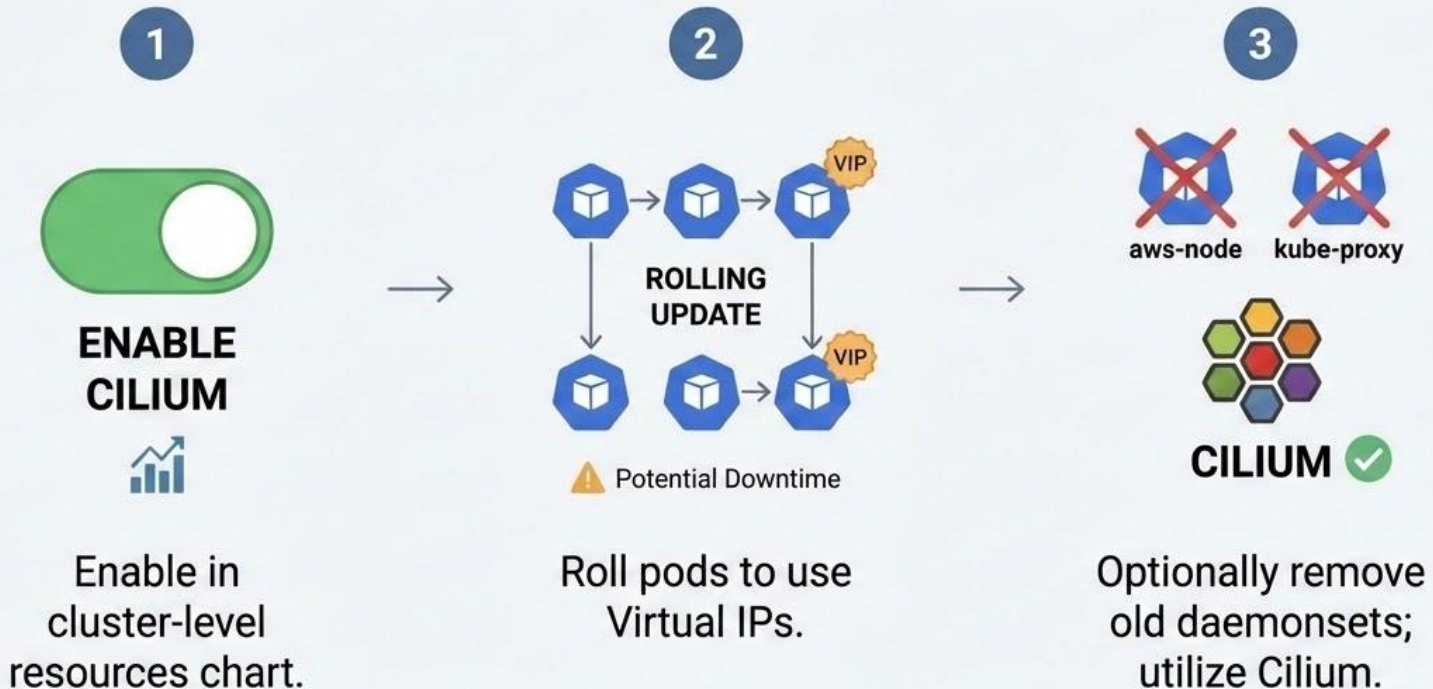


Enables a high Pod Limit of 2000/node, which is very unlikely to be reached.



This allows for Tighter Node Packing, running more pods per node efficiently.

Cilium - How to setup



Cilium - Next steps



More Features Beyond Virtual IPs



Cilium has a lot of other features we'd like to use and virtual IPs are just the start.



Hubble for Network Visibility



We can enable hubble to inspect network traffic.



Replace Calico with Cilium Policies



We can replace calico using the cilium network policies.



Service-to- Service TLS

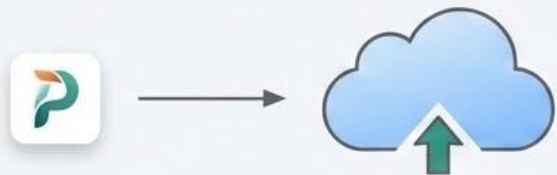


We can use cilium to setup service to service TLS.



The possibilities
are vast.

Shrinking Portal - The problem



High Memory at Startup (Build)

Portal is technically a **very lightweight** app that requires a lot of memory at startup to build the application.



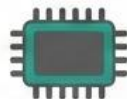
We need this because we need to build very different frontends based on any differences in the configuration files used, or even portal version.



Faster Builds = More CPU: We also want to have portal build slightly faster, so we end up giving more CPU which is only really needed at build time.



Failed Docker Image Approach: We tried to build individual docker images based on specific builds, but it became hard to manage and was not widely adopted.



Large Memory Requirement:
There is a need for a large amount of memory at build time, which ends up having us require a lot of memory.

Shrinking Portal - The solution



Cache Build Objects

Instead of building each time, cache build objects. Create new builds only if changes found.



Verify with Checksums

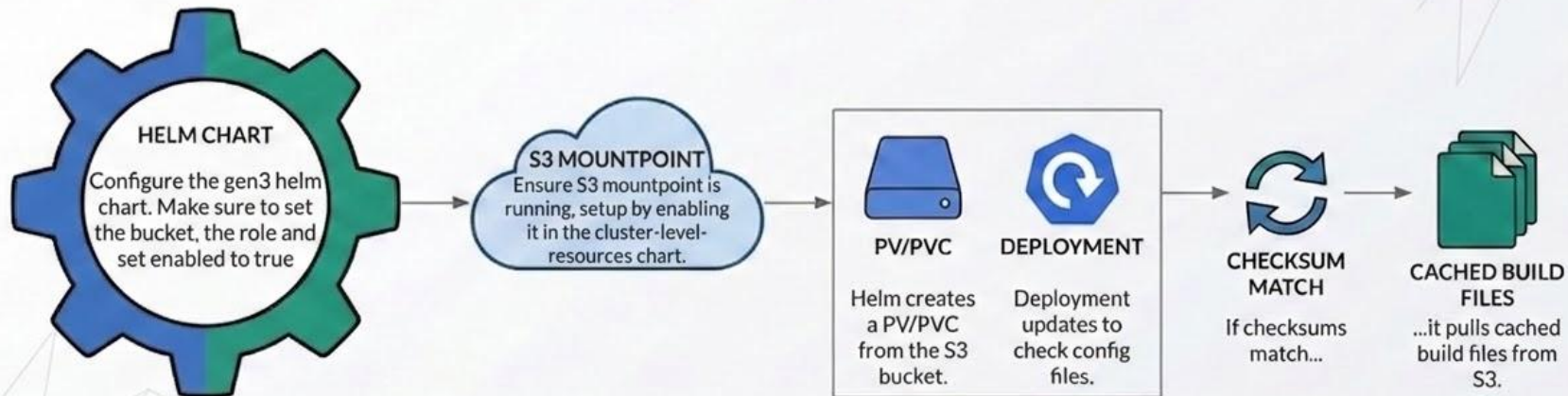
Utilize checksums of configuration to verify cached config. Pull from cache based on checksum.



S3 Mountpoint Backend

Build cache using mountpoint as backend. Allows multiple pods to use same PVC, not AZ-locked. Requires S3 mountpoint & portal role with bucket access.

Shrinking Portal - How to use it







Rethinking Stateful Services

Kubernetes vs. Managed Cloud Services

Core Question: Do our metrics and resource demands justify the premium price of managed services?
(Especially in dev/test/staging environments, lower tier prod environments)

Strategic Shift: Re-evaluating reliance on fully managed stateful services to drive down operational costs in lower tier environments.

Key Focus Areas: Elasticsearch and PostgreSQL.



Running PostgreSQL in Kubernetes

Single Pod:

- Bitnami Charts
- Easy to spin up.
- Good for dev/test - used in our CI!
- Out of support

Operator

- CloudnativePG
- <https://cloudnative-pg.io/>

Run PostgreSQL.

The Kubernetes way.

CloudNativePG is the Kubernetes operator that covers the full lifecycle of a highly available PostgreSQL database cluster with a primary/standby architecture, using native streaming replication.

[View on GitHub](#)

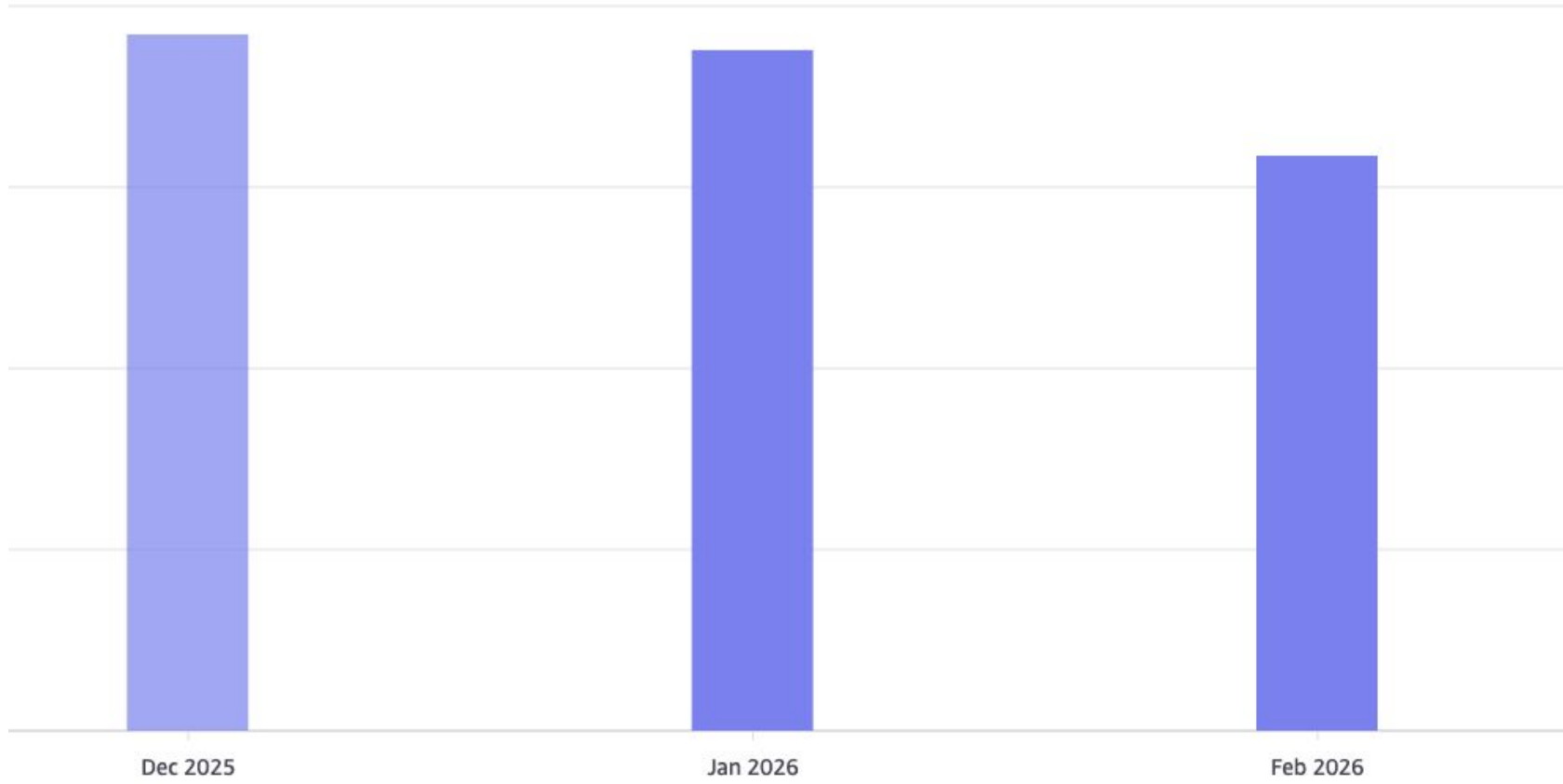






AWS Cost / usage reports

- How to analyze costs
- Find and clean up stale resources
 - Practice good cloud hygiene!





▼ Group by

Dimension

Clear

None



🔍 Filter groupings

None



Service

Linked account

Region

Instance type

Usage type

Resource

Cost category

Tag

API operation

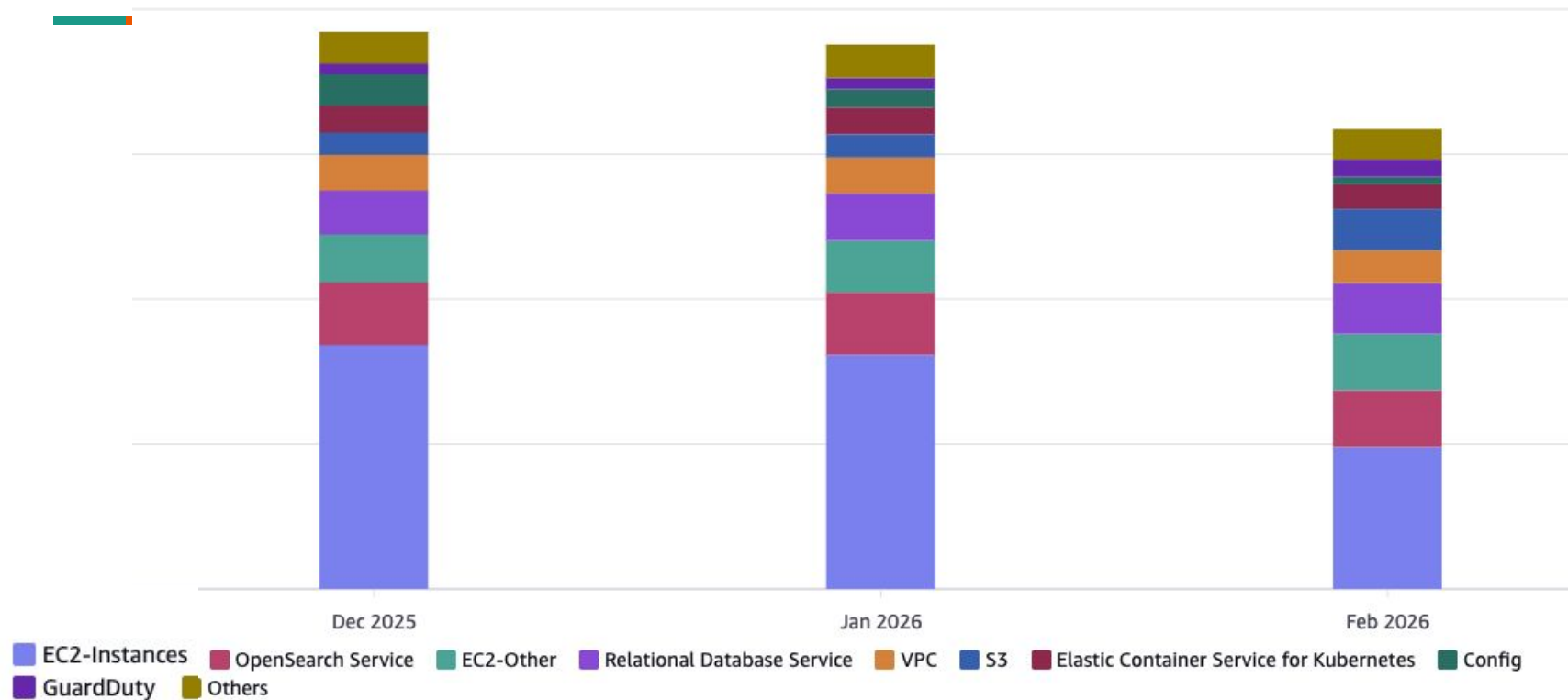
Availability zone

Platform

Purchase option

Usage type group

Clear



▼ Filters [Info](#)

Applied filters (0)

[Clear all](#) | [Settings](#)

Service [Clear](#)

Choose services



Includes



Excludes



Select all (2)



EC2 - Other



EC2-Instances (Elastic Compute Cloud - Compute)

Showing 2 of 39 results.

[Cancel](#)

[Apply](#)

▼ Group by

Dimension

[Clear](#)

Service

None

Service

Linked account

Region

Instance type

Usage type

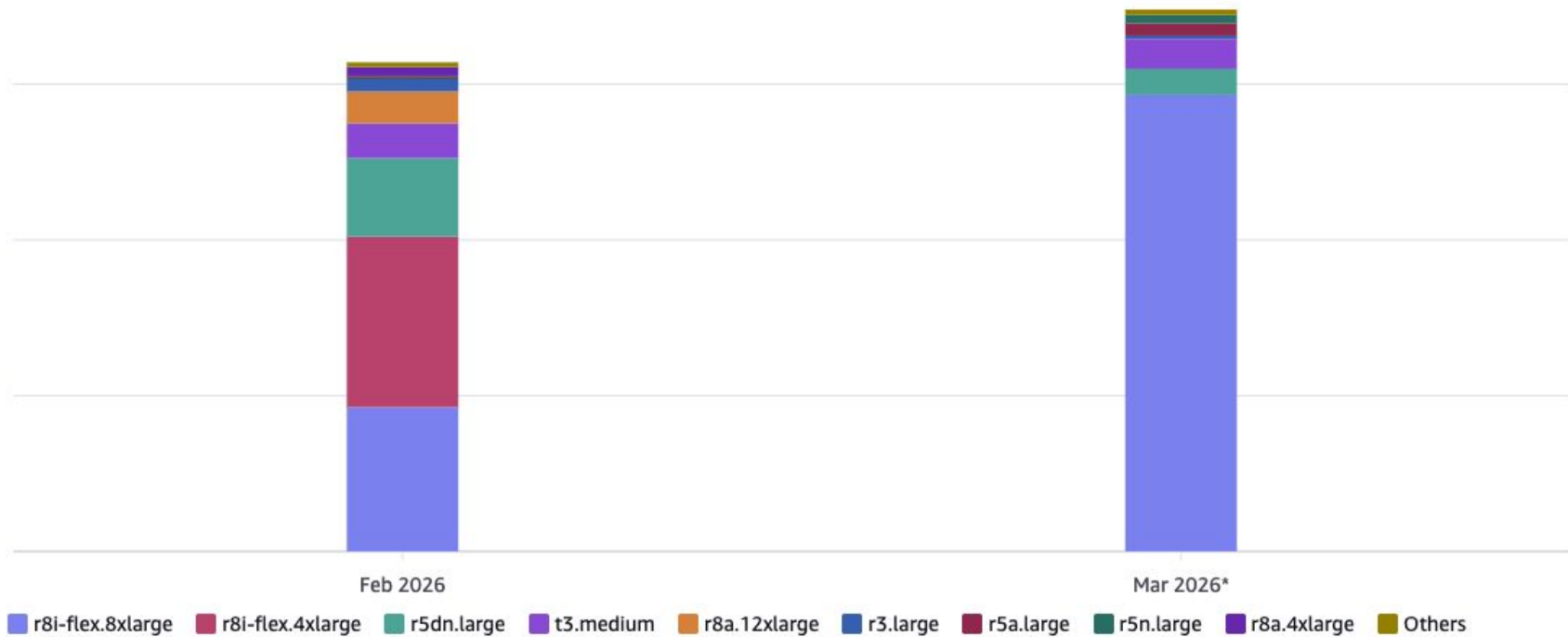
Resource

Cost category

Tag

API operation

Availability zone





Follow along?

- We will be making changes/updates to the cluster-level-resources charts in gen3-helm

Future:

- Graviton Instances with EKS 1.35 update
- Potentially dropping VPC endpoints



Q & A